JAVA PROGRAM REPORT

Thread Synchronization – Seat Reservation System

1. Objective / Aim

To demonstrate Java multithreading and synchronization by allowing multiple person threads to reserve seats from a shared reservation object without overselling the available seats.

2. System Requirements

Hardware: Standard PC/laptop, keyboard, monitor, and sufficient memory/storage.

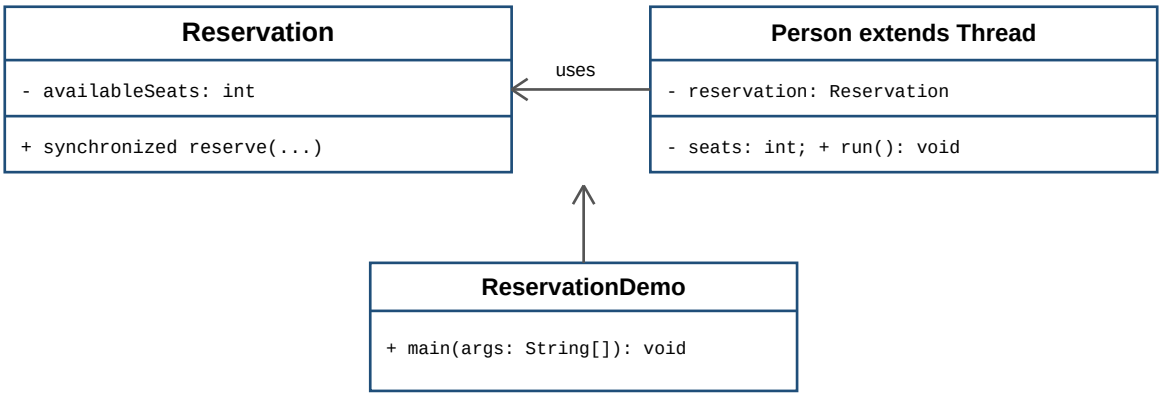
Software: Windows/Linux/macOS, JDK, and a Java IDE or command prompt/terminal.

3. Algorithm / Logic Description

1. Create a Reservation object with 10 available seats.
2. Define synchronized reserve(name, seats) to check and update the shared seat count atomically.
3. Create three Person threads requesting 5, 2, and 4 seats respectively.
4. Start Person-1 and wait for it to finish using join().
5. Start Person-2 and wait for it to finish.
6. Start Person-3 and wait for it to finish.
7. Print reservation status and remaining seats for each request.
8. Handle InterruptedException if a thread wait is interrupted.

4. UML Class Diagram

ReservationDemo creates Person threads that share a single Reservation object. Person extends Thread and calls the synchronized reserve method.



5. Source Code

```
class Reservation {
    int availableSeats = 10;

    synchronized void reserve(String name, int seats) {
        System.out.println(name + " entered.");
        System.out.println("Available seats: " + availableSeats
                + " Requested seats: " + seats);

        if (seats <= availableSeats) {
            System.out.println("Seat Available. Reserve now :-)");
            availableSeats = availableSeats - seats;
            System.out.println(seats + " seats reserved.");
        } else {
            System.out.println("Requested seats not available :-)");
        }

        System.out.println(name + " leaving.");
        System.out.println("----------------------------------------------");
    }
}

class Person extends Thread {
    Reservation reservation;
    int seats;
```

```
    Person(String name, Reservation reservation, int seats) {
        super(name);
        this.reservation = reservation;
        this.seats = seats;
    }

    public void run() {
        reservation.reserve(getName(), seats);
    }
}

public class ReservationDemo {
    public static void main(String[] args) {
        Reservation reservation = new Reservation();

        Person p1 = new Person("Person-1", reservation, 5);
        Person p2 = new Person("Person-2", reservation, 2);
        Person p3 = new Person("Person-3", reservation, 4);

        p1.start();
        try {
            p1.join();
            p2.start();
            p2.join();
            p3.start();
            p3.join();
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted.");
        }
    }
}
```

6. Compilation and Execution Commands

```
javac ReservationDemo.java
java ReservationDemo
```

7. Console Output Screenshot

The following console-style image was generated from the program's actual successful execution in this environment.

```
C:\Java-Projects> javac ReservationDemo.java
C:\Java-Projects> java ReservationDemo
Person-1 entered.
Available seats: 10 Requested seats: 5
Seat Available. Reserve now :-)
5 seats reserved.
Person-1 leaving.
--------------------------------------------
Person-2 entered.
Available seats: 5 Requested seats: 2
Seat Available. Reserve now :-)
2 seats reserved.
Person-2 leaving.
--------------------------------------------
Person-3 entered.
Available seats: 3 Requested seats: 4
Requested seats not available :-)
```

8. Explanation

• **synchronized:** Ensures only one thread at a time executes reserve() on the shared Reservation instance, protecting the seat count from concurrent updates.

• **Thread:** Person extends Thread so each person can run a reservation request as a thread.

• **join():** Makes the main thread wait for each person thread to finish before starting the next one. Thus, this particular demo executes the requests sequentially.

• **Shared object:** All three Person objects refer to the same Reservation object, so they use the same availableSeats value.

Code check: The program compiles and runs successfully. Starting and joining each thread in sequence means the output order is deterministic; Person-1 reserves 5 seats, Person-2 reserves 2, and Person-3's request for 4 seats is rejected because only 3 seats remain.